ACTIVITY 4

Database Users, Roles, Privileges, and Access Control

Securing a Database Environment

Course	COMP 018 – Database Administration	**Activity Type**	Group Activity
Database Platform	PostgreSQL	**Core Skill**	Role and privilege management

Activity Overview

Each group will set up the Activity 3 applications of all the other groups, which it finds through the class Google Sheets tracker. For each project, they must set up its PostgreSQL database, identify the different database access requirements, create appropriate database roles and login accounts, assign privileges, and test whether access restrictions work as intended.

The main principle is least privilege: every database account should receive only the permissions necessary for its intended responsibilities. Students must not use a PostgreSQL superuser or a single unrestricted account for ordinary application access.

I. Learning Objectives

1. Configure PostgreSQL databases for existing applications.
2. Differentiate database login roles, group roles, and administrative accounts.
3. Create database users and assign them to appropriate group roles.
4. Apply GRANT and REVOKE commands to manage database privileges.
5. Restrict access to databases, schemas, tables, sequences, and other required objects.
6. Test permitted and denied operations using different database accounts.
7. Document the database access-control structure and justify its security decisions.
8. Record the work in a PDF progress document and share the group's results through its own repository and the class Google Sheets tracker.

II. Activity Instructions

Part A – Database Setup and Assessment

Use the Activity 3 projects of all the other groups, as listed in the Google Sheets tracker (not your own project). Do not modify any owning group's repository; keep your own work in your group's repository (see Part F).

1. Open the Google Sheets tracker and clone the repository of every other group, then review each project's README, PDF progress documentation, user roles documentation, and database documentation.
2. Set up each application and its own PostgreSQL database using the documented procedures, keeping the databases separate.
3. Identify the database name, schemas, tables, sequences, views, and other database objects used by each application.

4. Review each application's database operations and its user roles documentation (from Activity 3) to determine which users or system components need access.
5. Prepare a database access-control plan for each project before creating users and assigning privileges.

Document any missing setup instructions or database inconsistencies encountered during this process, and report them to the owning group. If a project cannot be set up, record the blocking problem, report it to the owners, and continue with the remaining projects.

Part B – Identify Users, Group Roles, and Privileges

Determine the different types of database access required by the application.

Database role	Intended responsibilities	Example permissions
Database administrator	Provision and maintain the database environment	Administrative privileges appropriate to the task
Application runtime	Perform normal application operations	Required SELECT, INSERT, UPDATE, and DELETE permissions
Read-only user	View records for reporting or inspection	SELECT on approved tables or views
Data entry user	Add new records and view relevant data	INSERT and SELECT on approved tables
Auditor	Inspect records without modifying them	Read-only access to approved data

These are examples, not mandatory roles for every project. Identify appropriate roles based on each application's requirements and its documented user roles from Activity 3. A role's name does not enforce its intended responsibilities; implement and verify the actual PostgreSQL privileges.

Part C – Create Database Roles and Login Accounts

Use PostgreSQL role management commands to implement your access-control plan. Your implementation must include:

- **Group roles –** roles representing common sets of database privileges, generally created with NOLOGIN.
- **Login roles –** individual database accounts that can authenticate to PostgreSQL.
- **Role membership –** assign login roles to appropriate group roles.
- **Database access –** grant only the necessary database-level permissions.
- **Schema access –** grant the required schema privileges.
- **Object-level access –** configure table, sequence, view, and other necessary privileges.

Illustrative SQL example (adapt names and privileges to each project):

```
-- Create group roles
CREATE ROLE app_runtime_grp NOLOGIN;
CREATE ROLE app_readonly_grp NOLOGIN;

-- Create login roles
CREATE ROLE app_runtime LOGIN PASSWORD 'use-a-secure-password';
CREATE ROLE report_user LOGIN PASSWORD 'use-a-secure-password';

-- Assign group roles
GRANT app_runtime_grp TO app_runtime;
GRANT app_readonly_grp TO report_user;

-- Grant database connection access
GRANT CONNECT ON DATABASE project_db
```

```
TO app_runtime_grp, app_readonly_grp;

-- Grant schema access
GRANT USAGE ON SCHEMA public
TO app_runtime_grp, app_readonly_grp;

-- Grant table privileges
GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA public
TO app_runtime_grp;

GRANT SELECT
ON ALL TABLES IN SCHEMA public
TO app_readonly_grp;
```

This is an illustrative starting point, not a complete security configuration. Run database-level grants while connected to the appropriate administrative database, and schema/object grants while connected to the target database.

- PostgreSQL roles are cluster-wide, while database and schema privileges are scoped to their respective resources.
- CONNECT does not grant access to tables.
- USAGE on a schema does not automatically grant access to its tables.
- Table privileges do not automatically grant privileges on sequences used by operations such as inserts with generated identifiers.
- Privileges on existing objects do not automatically cover every future object. Review default privileges when necessary.
- Do not grant SUPERUSER, CREATEDB, CREATEROLE, or other elevated attributes unless specifically justified for an administrative account.

Never commit actual passwords to any repository, tracker sheet, or PDF document. Use secure, local configuration for credentials.

Part D – Implement the Principle of Least Privilege

Review default privileges and access permissions of each database you set up. Your group must ensure that:

1. The application does not use a PostgreSQL superuser account for normal operations.
2. Application accounts cannot access unrelated application databases merely because they were given broad privileges.
3. Users cannot modify tables or records outside their assigned responsibilities.
4. Read-only accounts cannot insert, update, or delete records.
5. Accounts do not receive unnecessary database creation or role-management privileges.
6. Publicly available privileges and inherited role memberships are reviewed and restricted where appropriate.

Do not assume that creating a separate username is sufficient. Examine the privileges the account actually inherits and the operations it can perform.

Part E – Test and Verify Access Control

Test each login account in every project database using a PostgreSQL client such as psql or an appropriate database administration tool.

Test operation	Expected result
Application runtime reads an authorized table	Allowed

Test operation	Expected result
Application runtime inserts an authorized record	Allowed, if required
Read-only user retrieves approved records	Allowed
Read-only user updates a record	Denied
Ordinary application user creates a new database	Denied
User accesses an unrelated database without permission	Denied, subject to other applicable access paths
User accesses a table without the required privileges	Denied

Adapt these tests to the privileges your group actually implements. For every test, record the account used, SQL statement or operation, expected result, and actual result, with a screenshot or output as evidence for the PDF progress documentation.

If an unauthorized operation succeeds, investigate the cause. Check direct grants, inherited roles, object ownership, PUBLIC privileges, and other applicable permissions before considering the configuration complete.

Part F – Repository, Tracker, and Progress Documentation

As in Activity 3, there is no centralized repository. Each group keeps its Activity 4 work in its own project repository and records its status in the class Google Sheets tracker. The group then submits a PDF that documents all of its progress.

Step 1 – Keep your work in your own repository

1. Create a folder such as db-access-control/ in your group's own repository, with one subfolder per project you set up (for example, db-access-control/group-03/) containing that project's SQL scripts, access-control plan, and test evidence.
2. Use meaningful commits so the repository history shows how the roles and privileges were developed.
3. Do not commit passwords or secrets. Use placeholders in scripts and keep real credentials in local configuration.
4. Do not change any other group's repository. Send recommended fixes to each owning group as feedback instead.

Step 2 – Update the Google Sheets tracker

Keep your group's Activity 3 row up to date. In addition, add a tab named Activity 4 Log to the tracker with one row for every project your group sets up, using the columns below. Edit only your own group's rows.

Activity 4 Log column	What to enter
Project	Group number and project name of the application you set up.
Repository cloned	Repository URL and the branch or tag used.
Database setup result	Success, Success with issues, or Failed, with a short note.
Access-control folder link	Link to that project's subfolder in your own repository.
Access-control status	Not started, Planning, Roles created, Privileges tested, or Completed.
Feedback sent to owners	Yes or No, with the date feedback was sent to the owning group.
PDF progress document	Link to your group's PDF progress documentation (same link on every row).

Activity 4 Log column	What to enter
Last updated	Date of the most recent update by your group.

Step 3 – Submit the PDF progress documentation

Continue the PDF progress document from Activity 3 by adding the sections below, or submit a separate PDF for this activity. Submit it together with the link to your repository.

Section	Contents
Cover and links	Group number, members, repository link, and tracker link.
Projects set up	Table of every project cloned, its repository link, its database objects, and its setup result.
Access-control plan	For each project, a table of each login account, group role, application role served, responsibility, and required privileges.
SQL scripts summary	Overview of the scripts for roles, memberships, grants, and restrictions, with the folder link.
Verification results	For each project, a table of each test: account, statement, expected result, actual result, and evidence.
Issues and fixes	Unexpected results found, their cause, and how they were corrected.
Feedback to owners	Setup problems and security recommendations sent to each owning group.
AI usage declaration	AI tools used, tasks performed with them, and how the output was validated.

The PDF must not contain real passwords or other secrets. Check screenshots to confirm that no credentials are visible.

III. Required Deliverables

- **Database setup evidence —** Proof that each project's database was successfully initialized.
- **Access-control plan —** For each project, a table describing each login account, group role, application role served, responsibility, and required privilege.
- **SQL scripts —** Scripts for creating roles, assigning memberships, granting privileges, and applying necessary restrictions, stored in the group's own repository.
- **Privilege verification report —** Test results for each project demonstrating both permitted and denied operations.
- **Database security documentation —** Explanation of the access-control structure and rationale for assigned privileges.
- **Progress documentation (PDF) —** A PDF recording all of the group's Activity 4 progress, submitted with the repository link and recorded in the Google Sheets tracker.
- **AI usage declaration —** AI tools used, if any, and how generated SQL and security configuration were validated.

Do not submit real passwords or other secrets in scripts, the repository, the tracker, or the PDF.

IV. Assessment Rubric

Criteria	Points
Database setup and assessment	5

Criteria	Points
Access-control planning and role design	10
Correct creation of roles and login accounts	15
Appropriate privilege assignment and least privilege	25
Successful verification of allowed and denied operations	20
Documentation and technical justification	10
Own repository, tracker update, and PDF progress documentation	10
AI usage transparency and validation	5
TOTAL	**100**

A configuration that merely creates users without testing their effective permissions should not receive full credit. The PDF progress documentation must be supported by the evidence and commit history in the group's own repository.

V. Submission Checklist

- ☐ All other groups' projects run with their own PostgreSQL databases.
- ☐ Database objects and access requirements have been identified for each project.
- ☐ Group roles and login roles are properly defined.
- ☐ Role memberships and database privileges are documented.
- ☐ Schema, table, and sequence permissions have been reviewed.
- ☐ Unnecessary elevated privileges have been avoided.
- ☐ Allowed and denied operations have been tested.
- ☐ SQL scripts and test evidence are included in the submission and the group's repository.
- ☐ The scripts are in the group's own repository, and the Activity 4 Log in the tracker is updated.
- ☐ The PDF progress documentation is complete and contains no secrets.
- ☐ No real passwords or secrets appear in any repository, tracker, or PDF.
- ☐ AI assistance and validation activities are declared.

Expected Outcome

By the end of Activity 4, each group should have working PostgreSQL databases for the projects it set up, each with a documented access-control structure, properly assigned user and group roles, and verified restrictions on database operations. These environments become the foundation for Activity 5 on database health, logs, maintenance, and performance.